

CS-860: ARTIFICIAL **INTELLIGENCE**

- **Informed Search**
 - **A* Search**

**Text : Artificial Intelligence a Modern Approach
by Stuart Russel and Peter Norvig**

INSTRUCTOR:

DR. KUNWAR FARAZ AHMED KHAN

Informed Search



- Informed Search
 - A* Search

A* Search



UCS



Greedy



A*



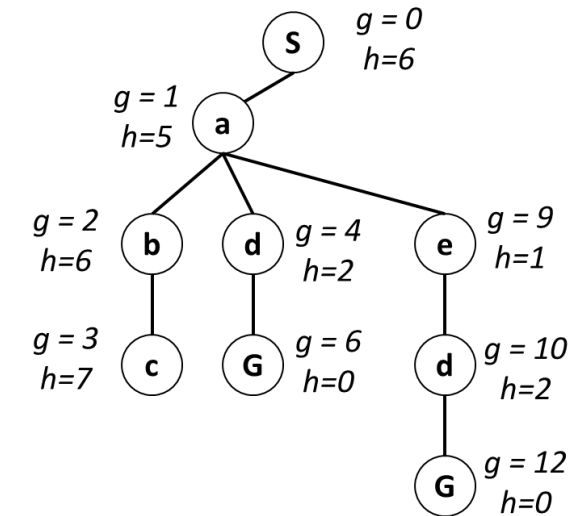
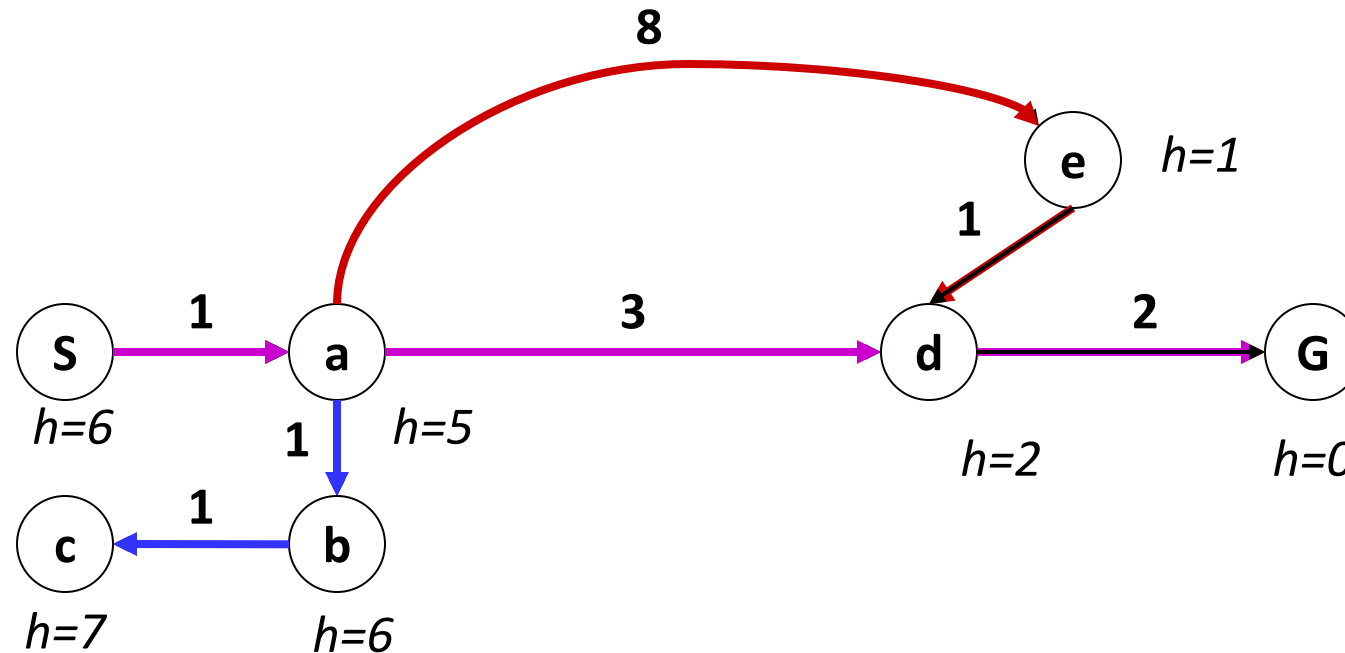
Informed Search



- Informed Search
 - A* Search

Combining UCS and Greedy

- Uniform-cost
- Greedy



- A* Search



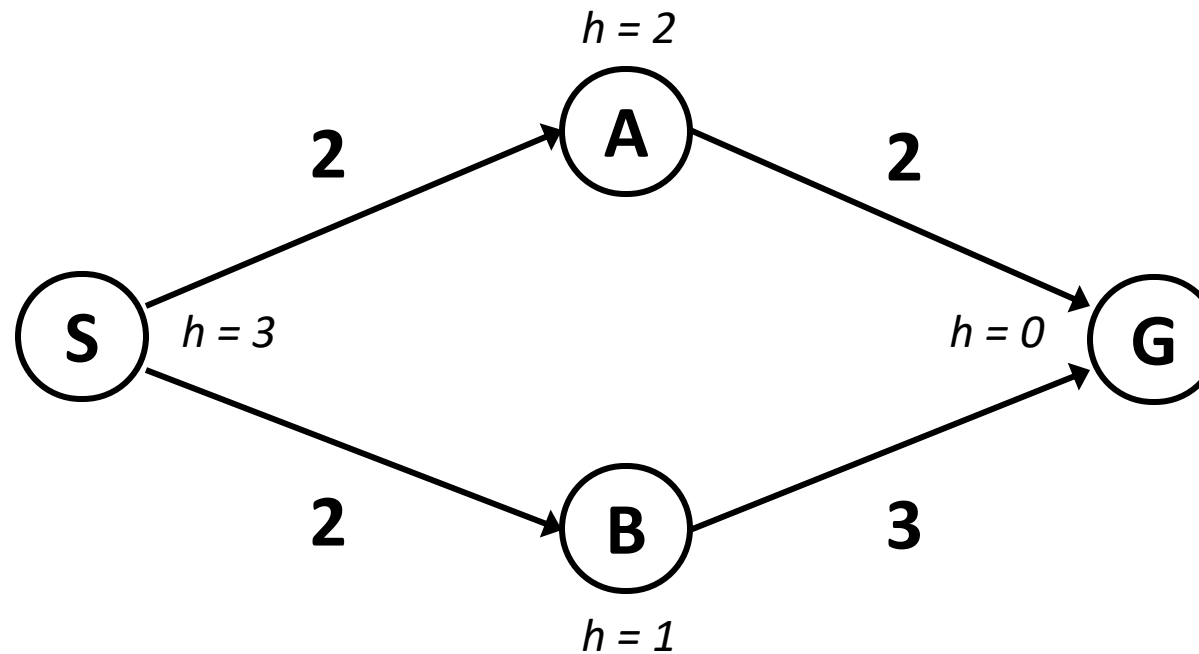
Informed Search



- Informed Search
 - A* Search

When Should A* Terminate

- Should we stop when we en-queue a goal (i.e., add to fringe)?



Order in which we add to fringe:

S

S→A

S→B

Expand B first

S→B→G (cost 5)

Don't declare victory yet! Be patient!

S→A→G (cost 4)

Victory!

- No: only stop when we de-queue a goal



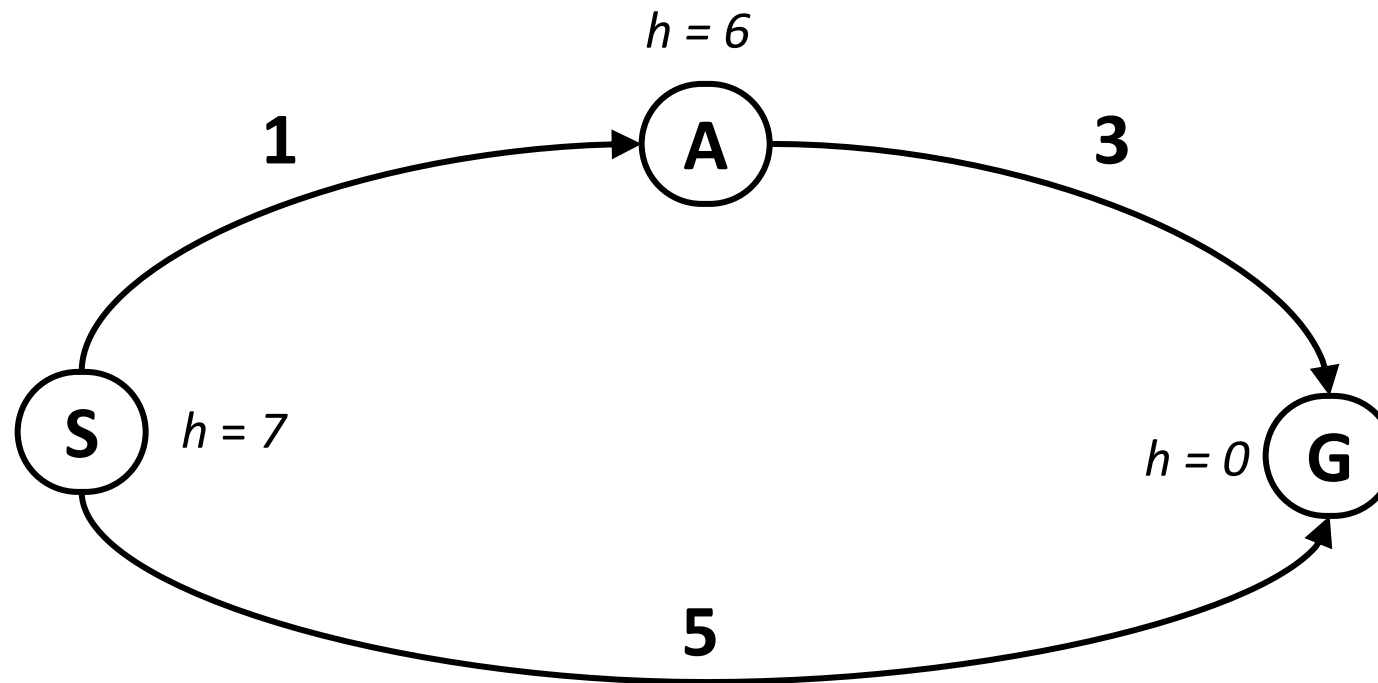
Informed Search



- Informed Search
 - A* Search

Is A* Optimal

- What went wrong?
 - Actual bad goal cost < estimated good goal cost
 - We need estimates to be less than actual costs!



Order in which we add to fringe:

S

S→A ($g + h = 7$)

S→G ($g + h = 5$)

Goal check:

S→G passes. Done!

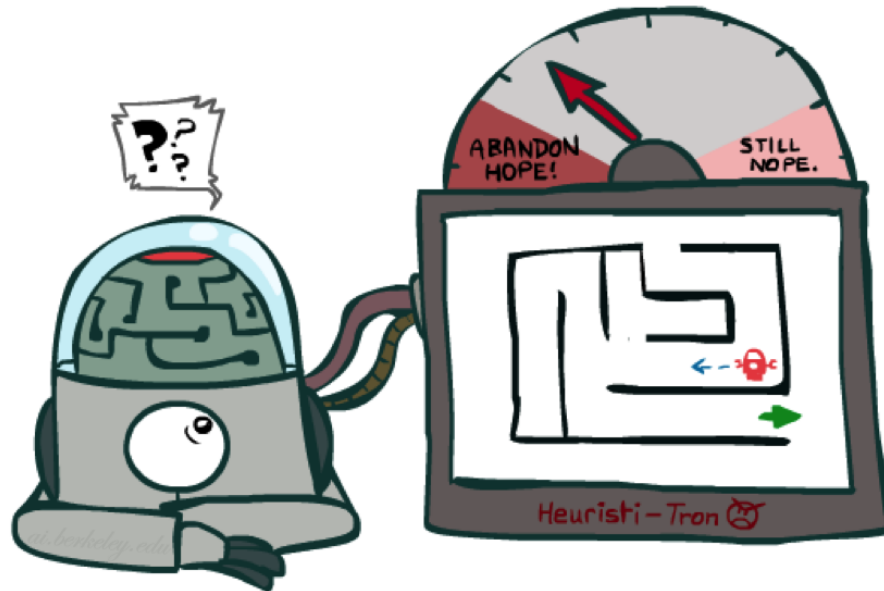


Informed Search

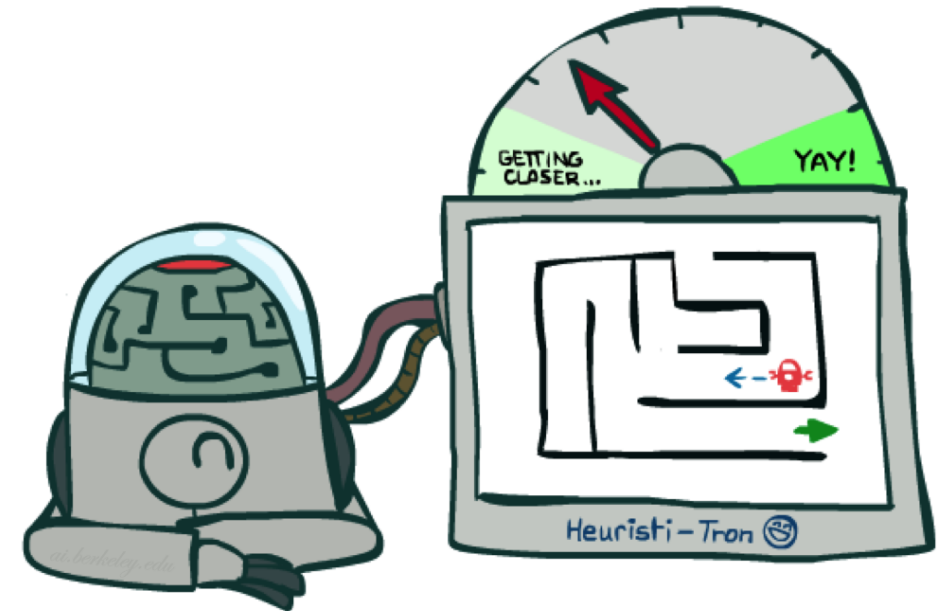


- **Informed Search**
 - A* Search
 - **Admissible Heuristics**

Admissible Heuristics



Inadmissible (pessimistic) heuristics break optimality by trapping good plans on the fringe



Admissible (optimistic) heuristics slow down bad plans but never outweigh true costs



Informed Search



- Informed Search
 - A* Search
 - Admissible Heuristics

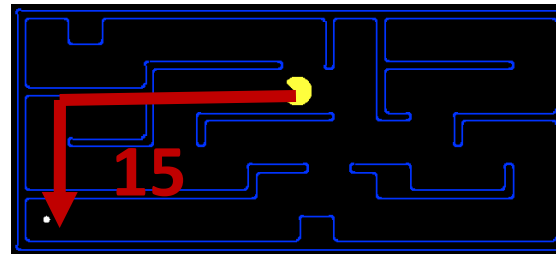
Admissible Heuristics

- A heuristic h is *admissible* (optimistic) if:

$$0 \leq h(n) \leq h^*(n)$$

where $h^*(n)$ is the true cost to a nearest goal

- Examples:



4



- Coming up with admissible heuristics is most of what's involved in using A* in practice.



Informed Search



- **Informed Search**

- A* Search
- Admissible Heuristics
- **Optimality of A* Tree Search**

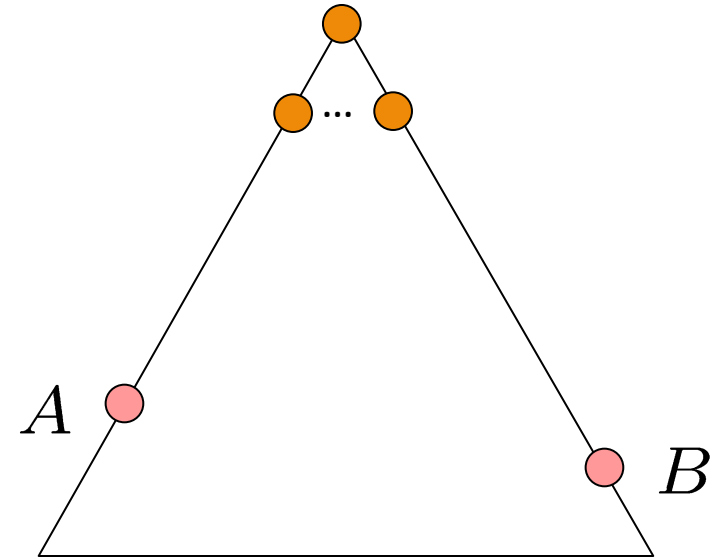
Optimality of A* Tree Search

Assume:

- A is an optimal goal node
- B is a suboptimal goal node
- h is admissible

Claim:

- A will exit the fringe before B



Informed Search

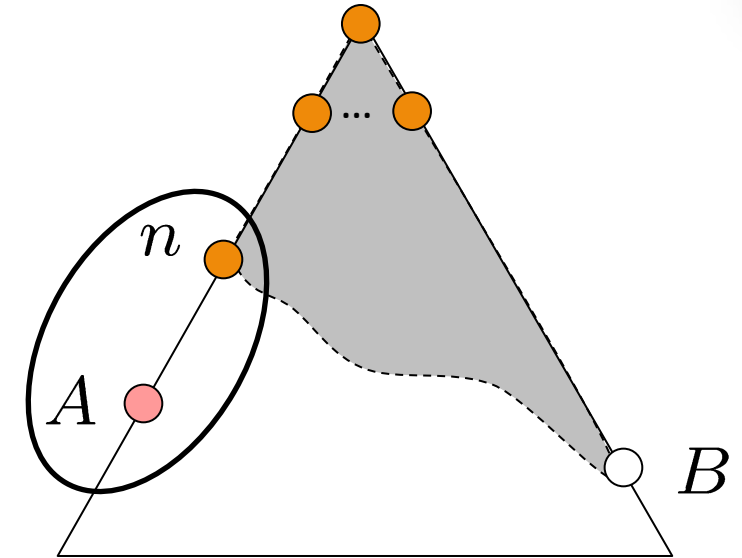


- Informed Search
 - A* Search
 - Admissible Heuristics
 - Optimality of A* Tree Search

Optimality of A* Tree Search – Blocking

Proof:

- Imagine B is on the fringe
- Some ancestor n of A is on the fringe, too (maybe A !)
- Claim: n will be expanded before B
 1. $f(n)$ is less or equal to $f(A)$



$$f(n) = g(n) + h(n)$$

Definition of f-cost

$$f(n) \leq g(A)$$

Admissibility of h

$$g(A) = f(A)$$

$h = 0$ at a goal



Informed Search

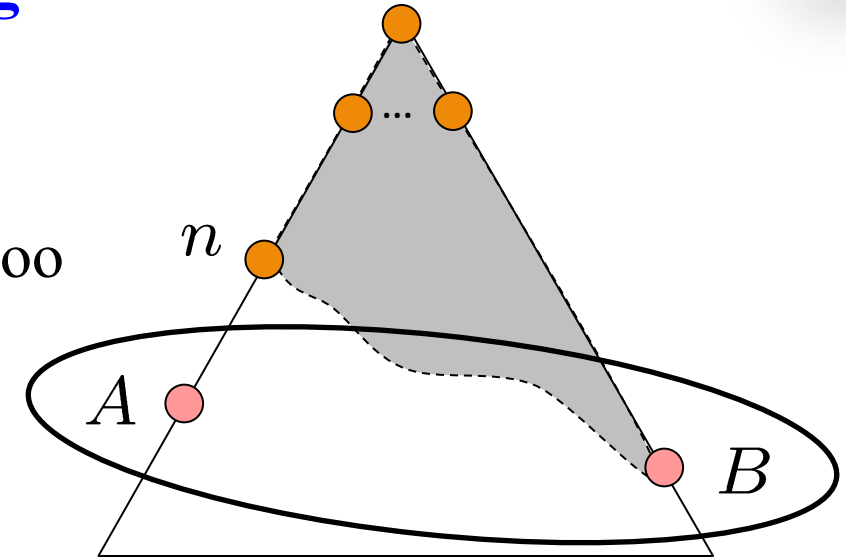


- **Informed Search**
 - A* Search
 - Admissible Heuristics
 - **Optimality of A* Tree Search**

Optimality of A* Tree Search – Blocking

Proof:

- Imagine B is on the fringe
- Some ancestor n of A is on the fringe, too (maybe A !)
- Claim: n will be expanded before B
 1. $f(n)$ is less or equal to $f(A)$
 2. $f(A)$ is less than $f(B)$



$$g(A) < g(B)$$

$$f(A) < f(B)$$

B is suboptimal

$h = 0$ at a goal



Informed Search



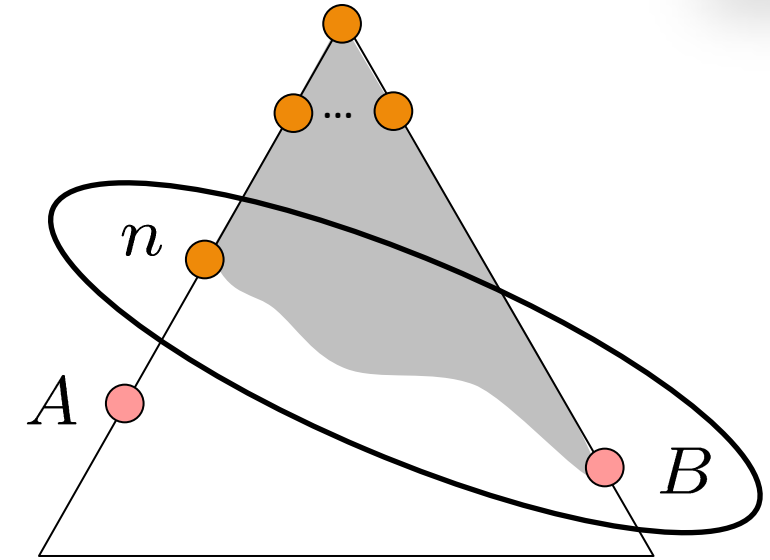
- **Informed Search**

- A* Search
- Admissible Heuristics
- **Optimality of A* Tree Search**

Optimality of A* Tree Search – Blocking

Proof:

- Imagine B is on the fringe
- Some ancestor n of A is on the fringe, too (maybe A !)
- Claim: n will be expanded before B
 1. $f(n)$ is less or equal to $f(A)$
 2. $f(A)$ is less than $f(B)$
 3. n expands before B
- All ancestors of A expand before B
- A expands before B
- A* search is optimal



$$f(n) \leq f(A) < f(B)$$



Informed Search

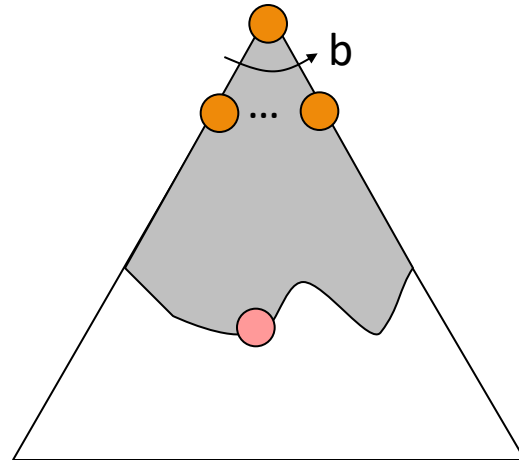


- **Informed Search**

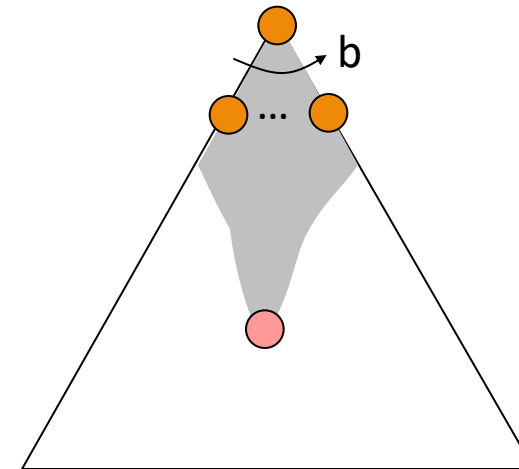
- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- **Properties of A* Search**

Properties of A* Search

Uniform-Cost



A*



Informed Search

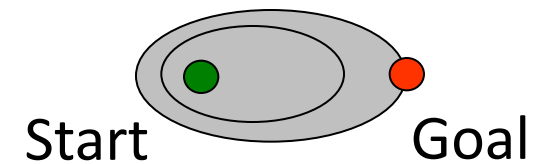
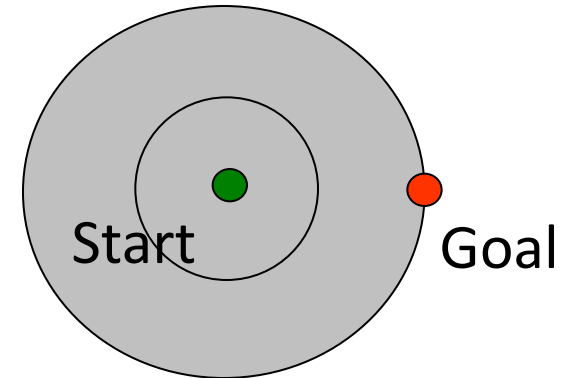


- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- **UCS vs A* Search**

UCS vs A* Contours

- Uniform-cost expands equally in all “directions”
- A* expands mainly toward the goal, but does hedge its bets to ensure optimality



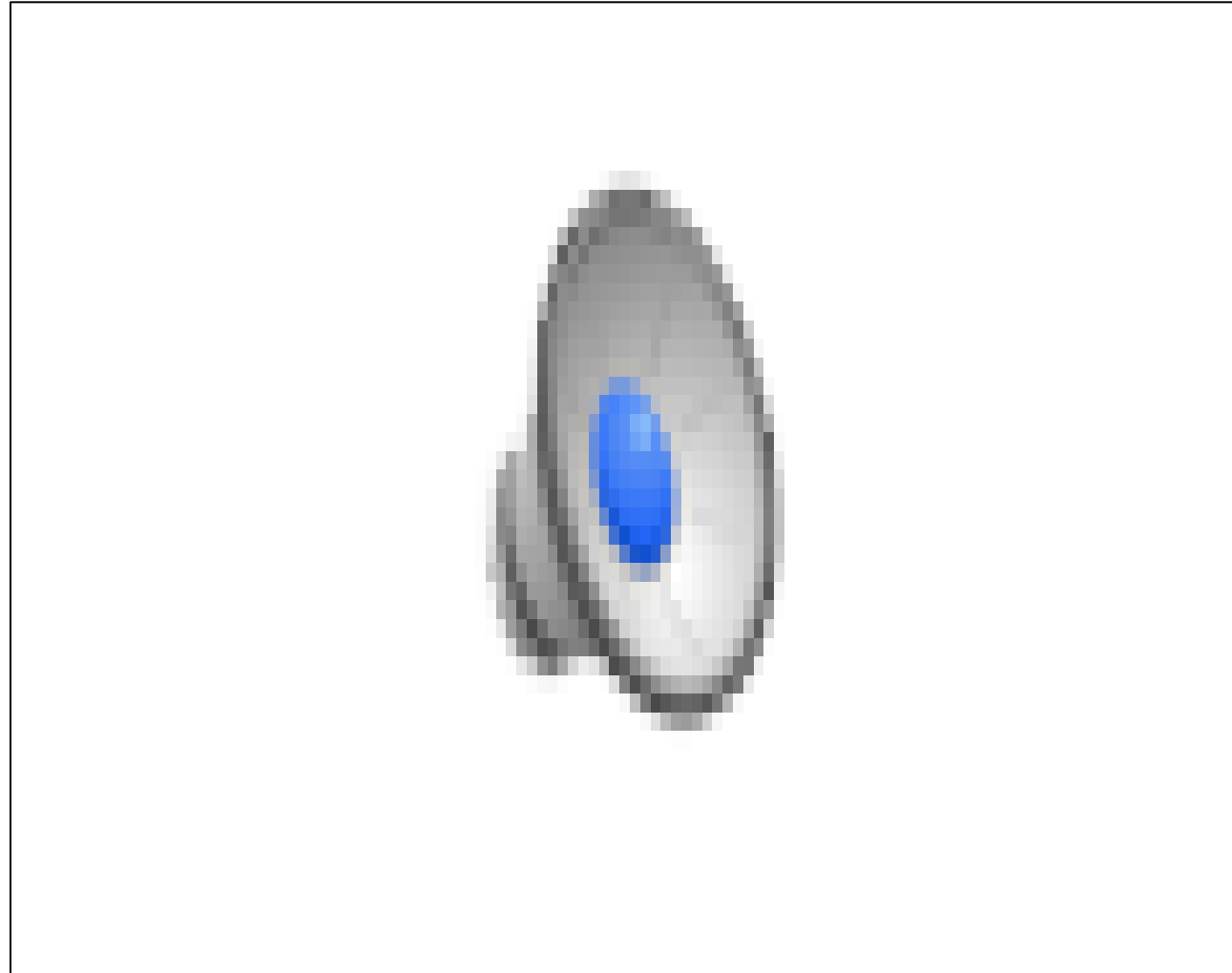
Informed Search



- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- **Demo**

Video of Demo Contours (Empty) -- UCS



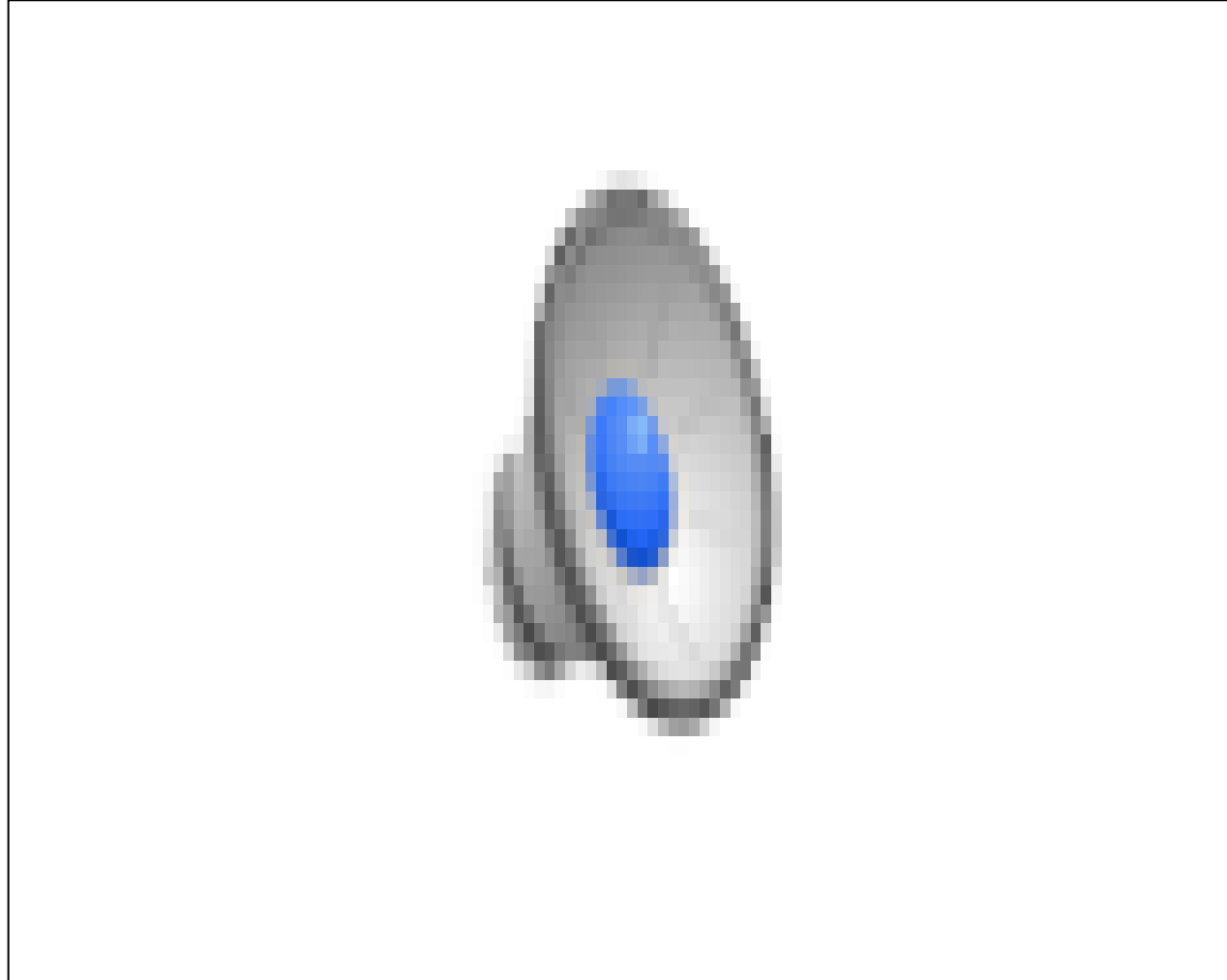
Informed Search



- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- **Demo**

Video of Demo Contours (Empty) -- Greedy



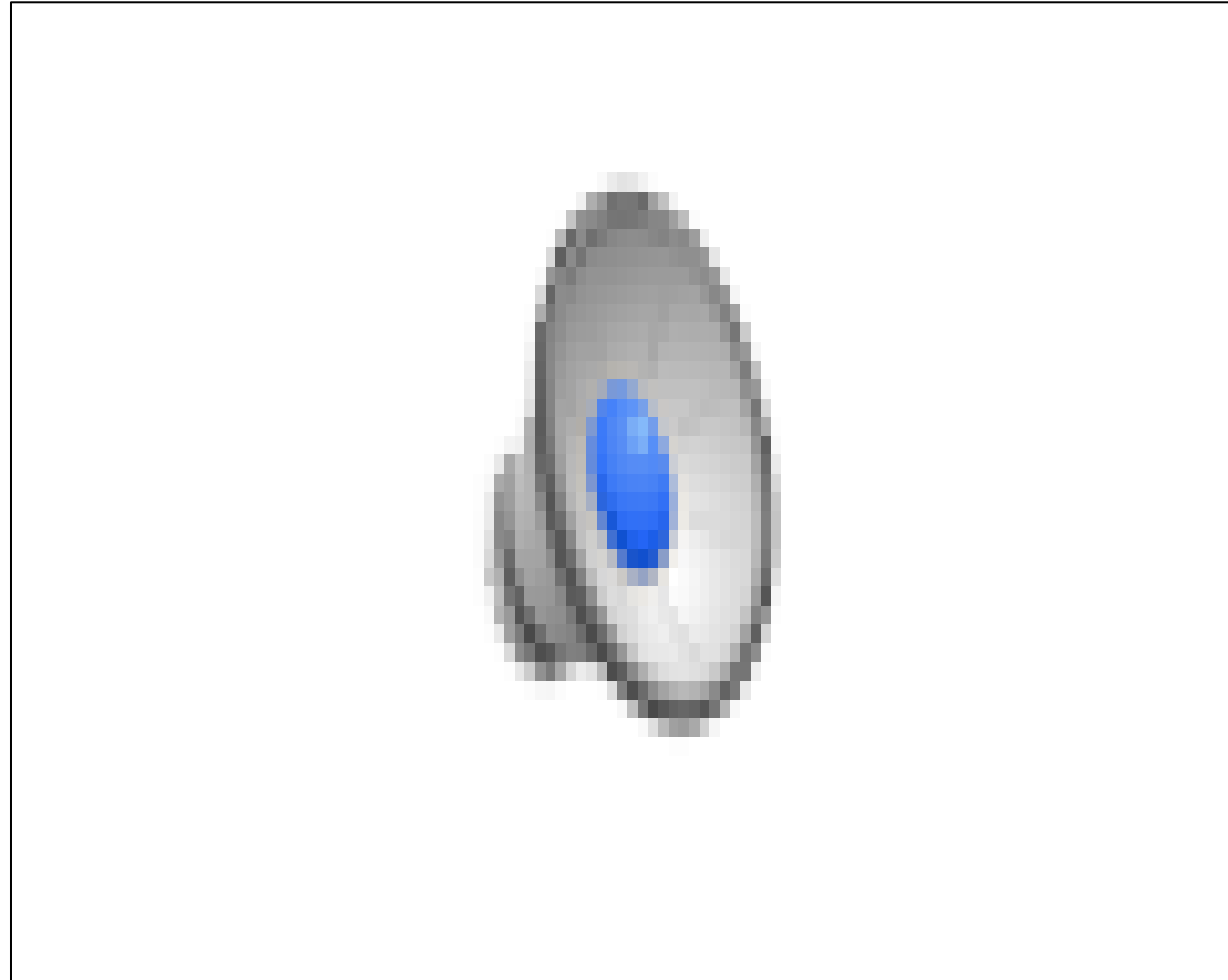
Informed Search



- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- **Demo**

Video of Demo Contours (Empty) – A* Search



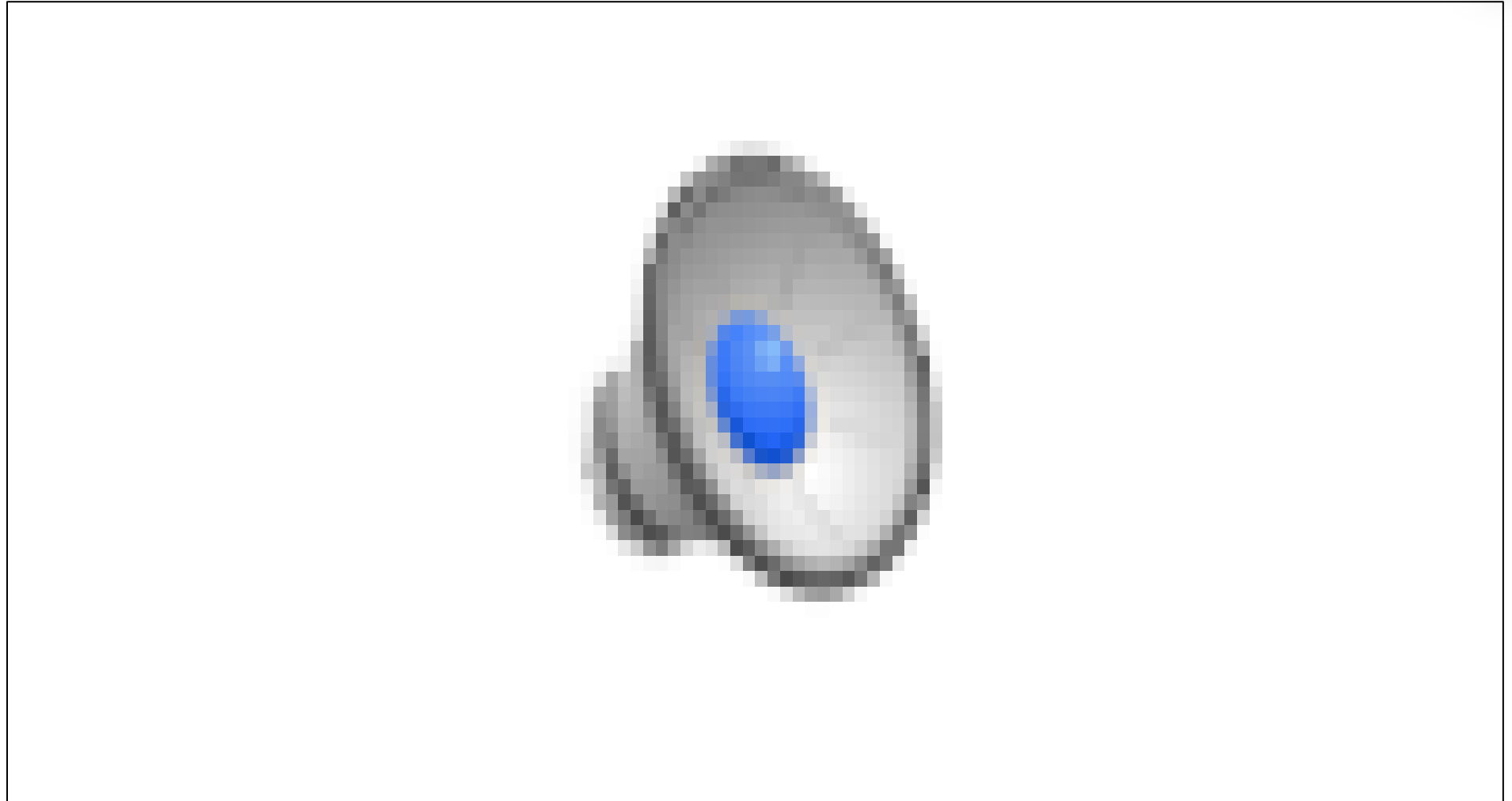
Informed Search



- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- **Demo**

Video of Demo Contours (Packman Small Maze) – A* Search



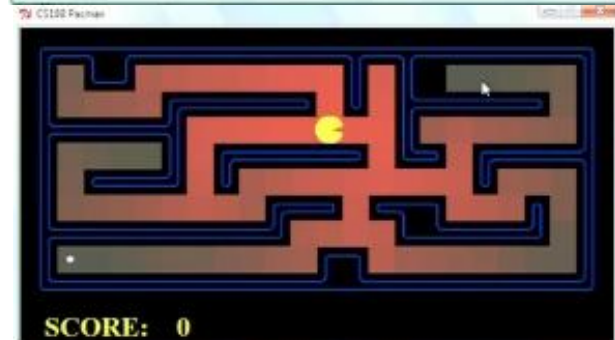
Informed Search



Comparison



Greedy



Uniform Cost



A*



Informed Search



- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- **Demo**

A* Applications

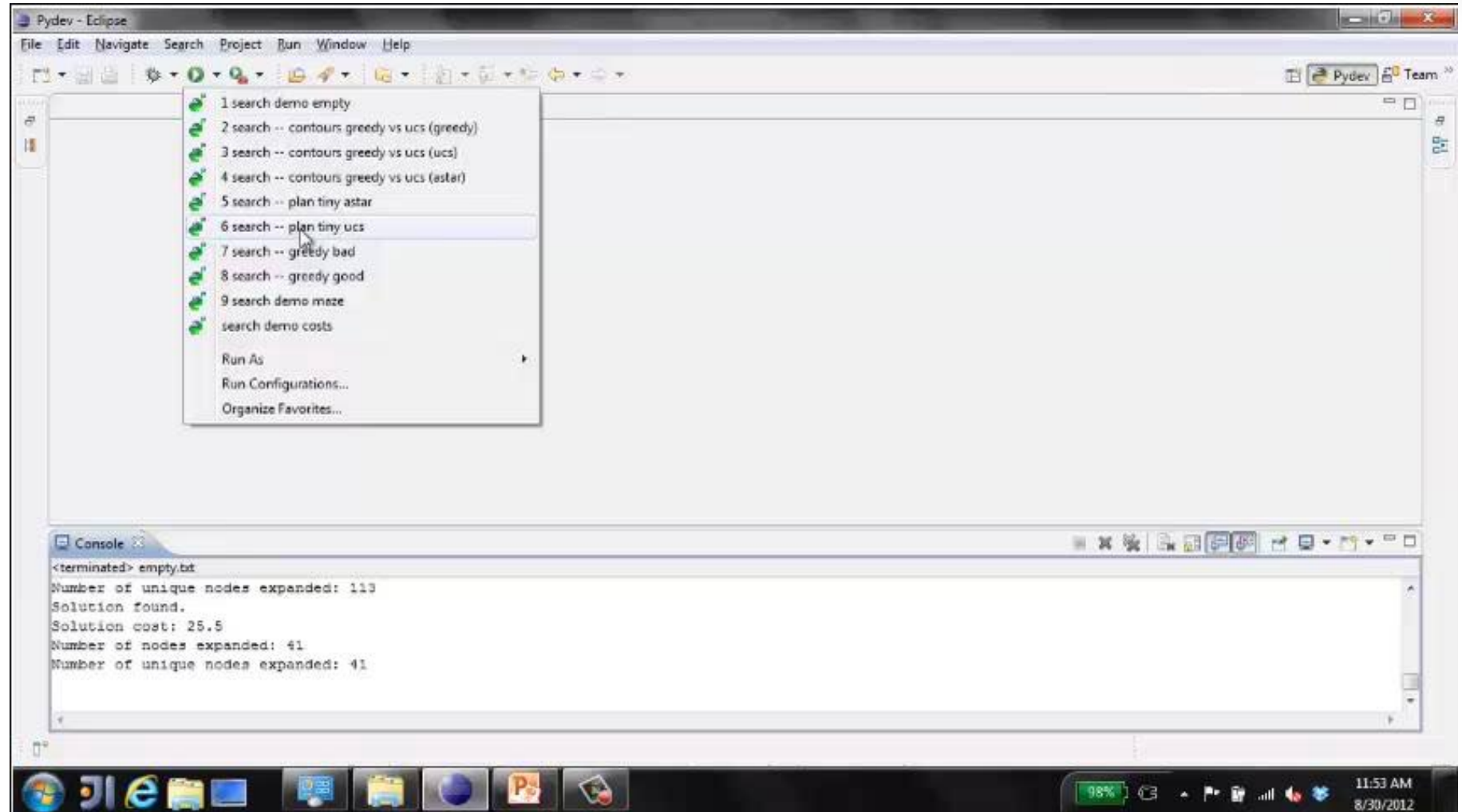
- Video games
- Pathing / routing problems
- Resource planning problems
- Robot motion planning
- Language analysis
- Machine translation
- Speech recognition
- ...



Informed Search



Video of Demo Pacman (Tiny Maze) – UCS / A*



Informed Search



Video of Demo Empty Water Shallow/Deep – Guess Algorithm

```
1 search -- plan tiny astar
2 search -- plan tiny ucs
3 search demo empty
4 search -- contours greedy vs ucs (greedy)
5 search -- contours greedy vs ucs (ucs)
6 search -- contours greedy vs ucs (astar)
7 search -- greedy bad
8 search -- greedy good
9 search demo maze
search demo costs
Run As
Run Configurations...
Organize Favorites...
```

```
<terminated> 1.5
Total cost: 27
Number of nodes expanded: 182
Number of unique nodes expanded: 182
Pacman emerges victorious! Score: 573
{'numKills': [0], 'results': ['Win'], 'numMoves': [27], 'scores': [573]}
```

- **Informed Search**
 - A* Search
 - Admissible Heuristics
 - Optimality of A* Tree Search
 - Properties of A* Search
 - UCS vs A* Search
 - Demo



Informed Search

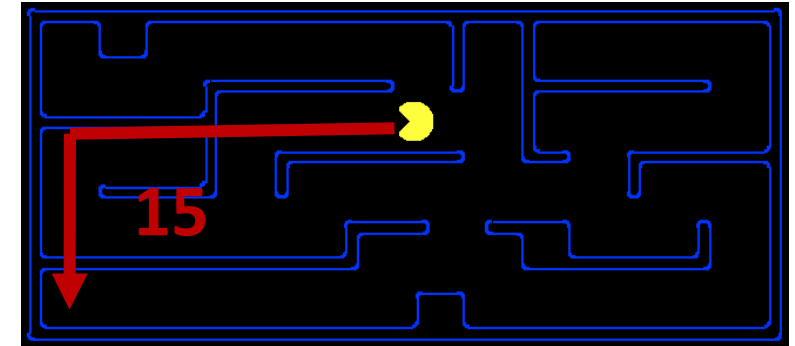
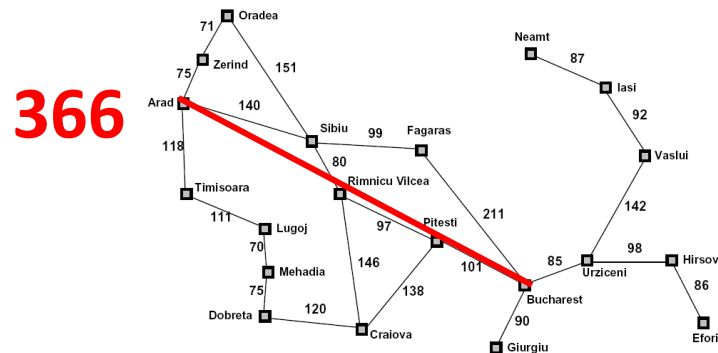


- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- Demo
- **Creating Heuristics**

Creating Admissible Heuristics

- Most of the work in solving hard search problems optimally is coming up with admissible heuristics
- Often, admissible heuristics are solutions to *relaxed problems*, where new actions are available



- Inadmissible heuristics are often useful too (find solutions fast)



Informed Search



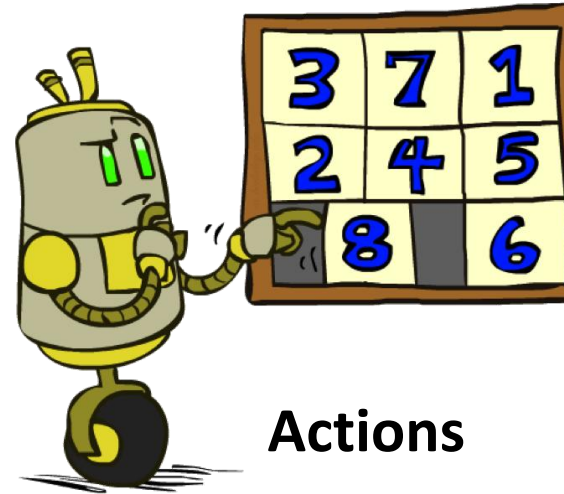
- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- Demo
- **Creating Heuristics**
 - Examples – 8 Puzzle

Example – 8 Puzzle

7	2	4
5		6
8	3	1

Start State



Actions

	1	2
3	4	5
6	7	8

Goal State

- What are the states?
- How many states?
- What are the actions?
- How many successors from the start state?
- What should the costs be?
- What are possible heuristics?



Informed Search

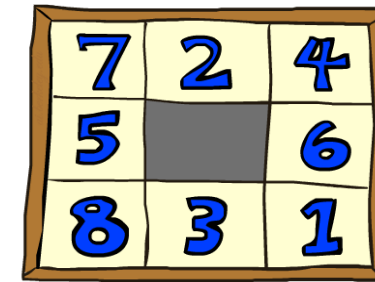


- **Informed Search**

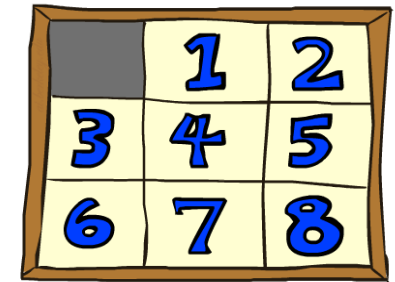
- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- Demo
- **Creating Heuristics**
 - Examples – 8 Puzzle

Example – 8 Puzzle – I

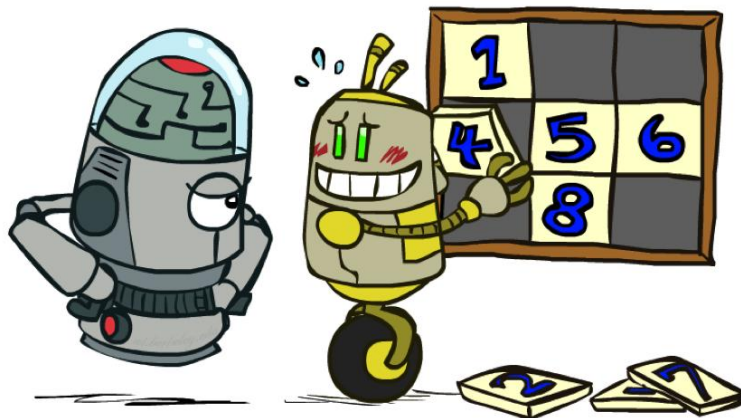
- Heuristic: Number of tiles misplaced
- Why is it admissible?
- $h(\text{start}) = 8$
- This is *relaxed-problem* heuristic



Start State



Goal State



	Average nodes expanded when the optimal path has...		
	...4 steps	...8 steps	...12 steps
UCS	112	6,300	3.6×10^6
TILES	13	39	227



Informed Search

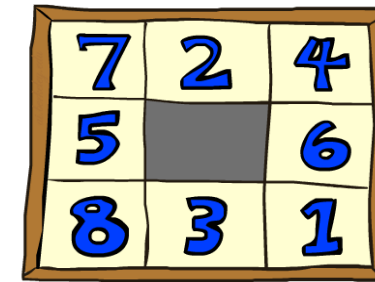


- **Informed Search**

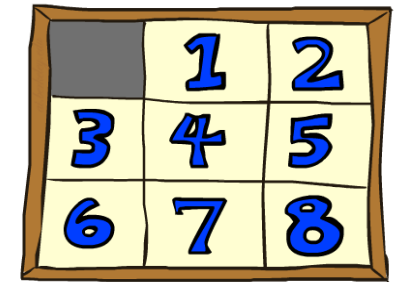
- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- Demo
- **Creating Heuristics**
 - Examples – 8 Puzzle

Example – 8 Puzzle – II

- What if we had an easier 8-puzzle where any tile could slide any direction at any time, ignoring other tiles?



Start State



Goal State

- Total *Manhattan* distance

- Why is it admissible?

- $h(\text{start}) =$

$$3 + 1 + 2 + \dots = 18$$

	Average nodes expanded when the optimal path has...		
	...4 steps	...8 steps	...12 steps
TILES	13	39	227
MANHATTAN	12	25	73



Informed Search

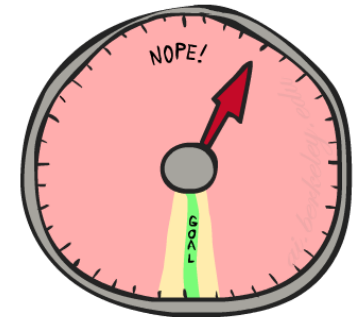
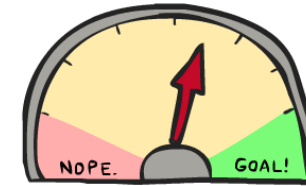


- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- Demo
- **Creating Heuristics**
 - Examples – 8 Puzzle

Example – 8 Puzzle – I

- How about using the *actual cost* as a heuristic?
 - Would it be admissible?
 - Would we save on nodes expanded?
 - What's wrong with it?



- With A*: a trade-off between quality of estimate and work per node
 - As heuristics get closer to the true cost, you will expand fewer nodes but usually do more work per node to compute the heuristic itself



Informed Search



- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- Demo

- **Creating Heuristics**

- Examples – 8 Puzzle
- Trivial Heuristics Dominance

Trivial Heuristics Dominance

- Dominance: $h_a \geq h_c$ if

$$\forall n : h_a(n) \geq h_c(n)$$

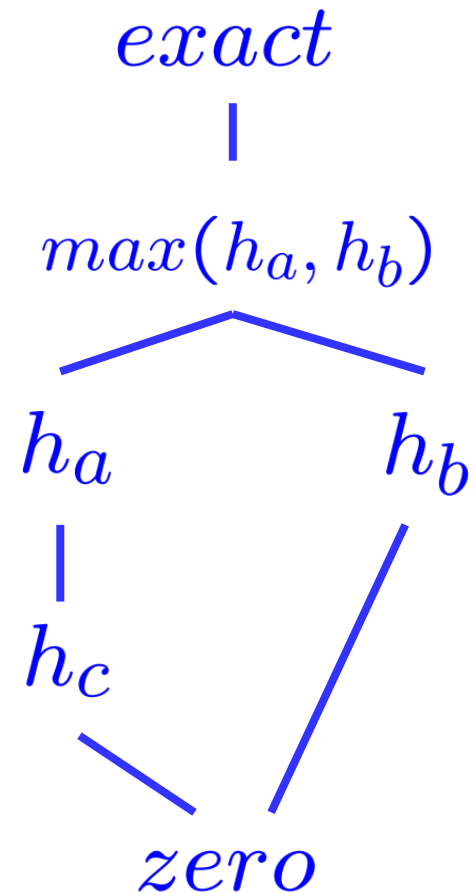
- Heuristics form a semi-lattice:

- Max of admissible heuristics is admissible

$$h(n) = \max(h_a(n), h_b(n))$$

- Trivial heuristics

- Bottom of lattice is the zero heuristic (what does this give us?)
- Top of lattice is the exact heuristic



Informed Search



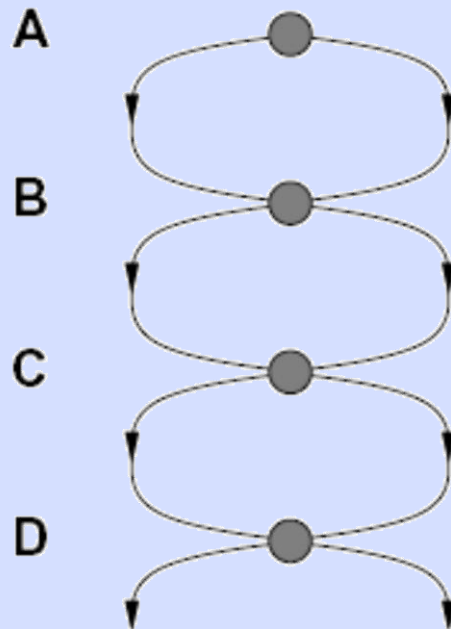
- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- Demo
- Creating Heuristics
- **Tree Search**

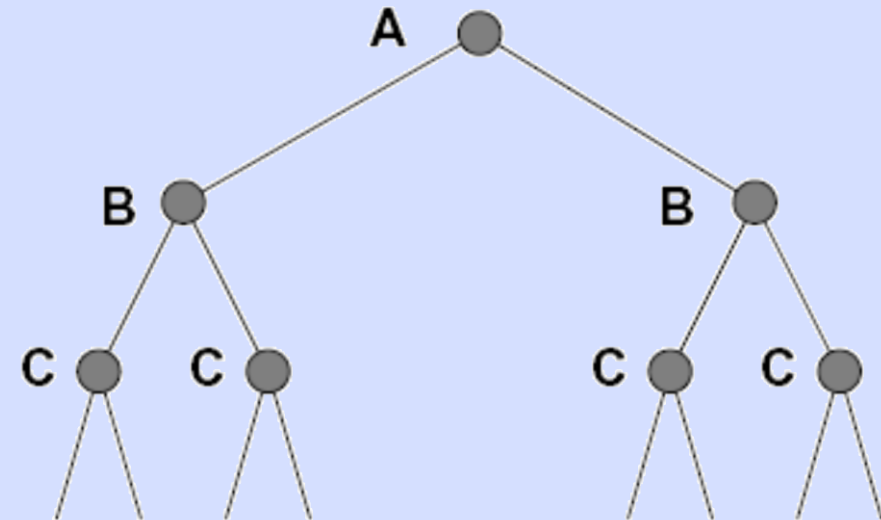
Tree Search Extra Work

- Failure to detect repeated states can cause exponentially more work.

State Graph



Search Tree



Informed Search

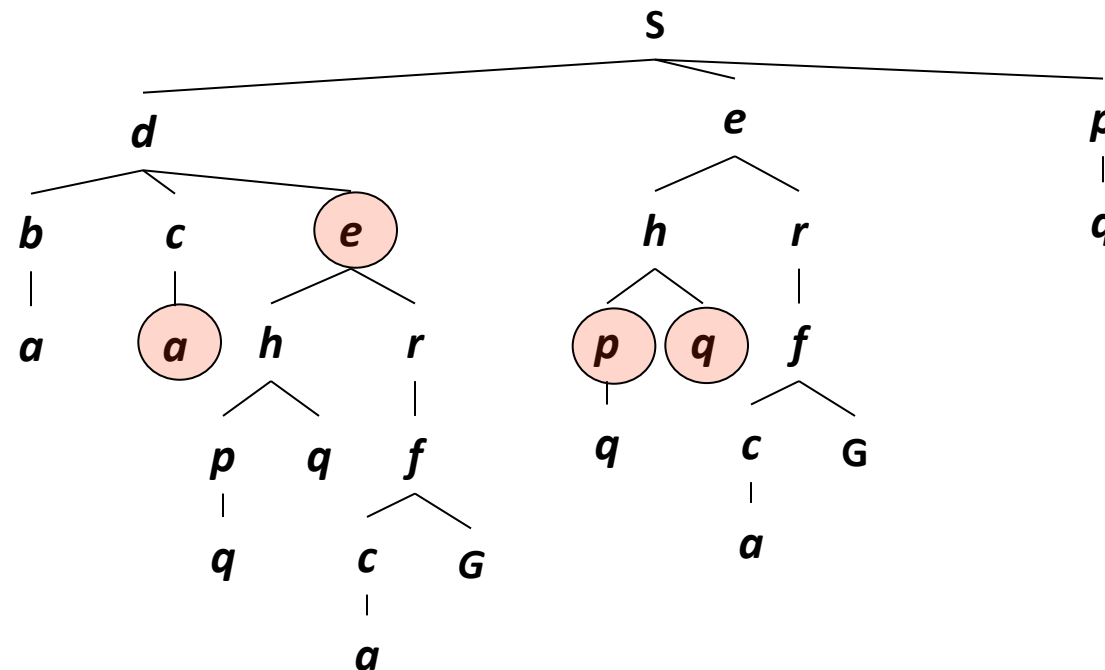


- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- Demo
- Creating Heuristics
- Tree Search
- **Graph Search**

Graph Search

- In BFS, for example, we shouldn't bother expanding the circled nodes (why?)



Informed Search



- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- Demo
- Creating Heuristics
- Tree Search
- **Graph Search**

Graph Search

- Idea: never **expand** a state twice
- How to implement:
 - Tree search + set of expanded states (“closed set”)
 - Expand the search tree node-by-node, but...
 - Before expanding a node, check to make sure its state has never been expanded before
 - If not new, skip it, if new add to closed set
- Important: **store the closed set as a set**, not a list
- Can graph search wreck completeness? Why/why not?
- How about optimality?



Informed Search

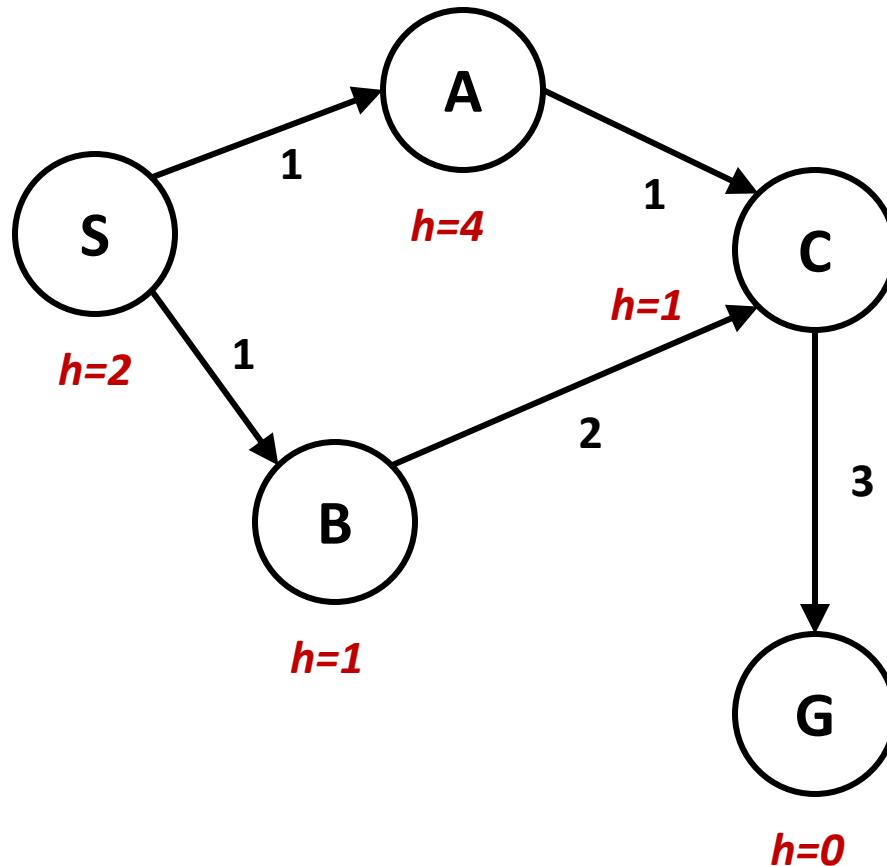


- **Informed Search**

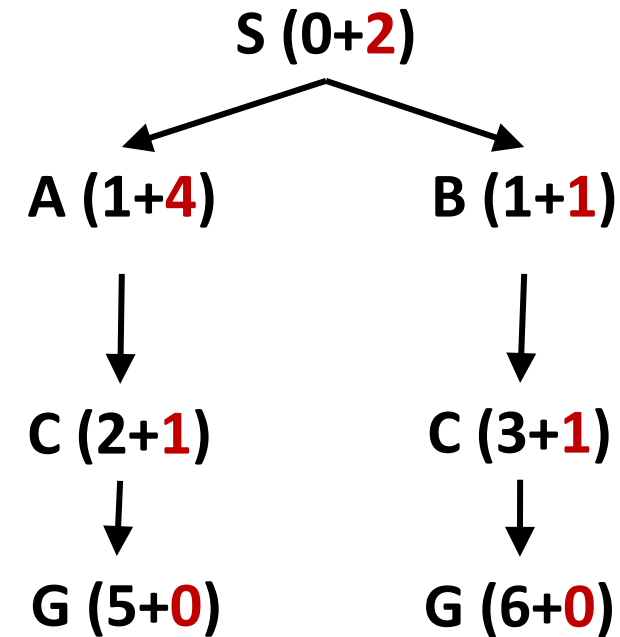
- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- Demo
- Creating Heuristics
- Tree Search
- **Graph Search**

A* Graph Search Gone Wrong

State space graph



Search tree



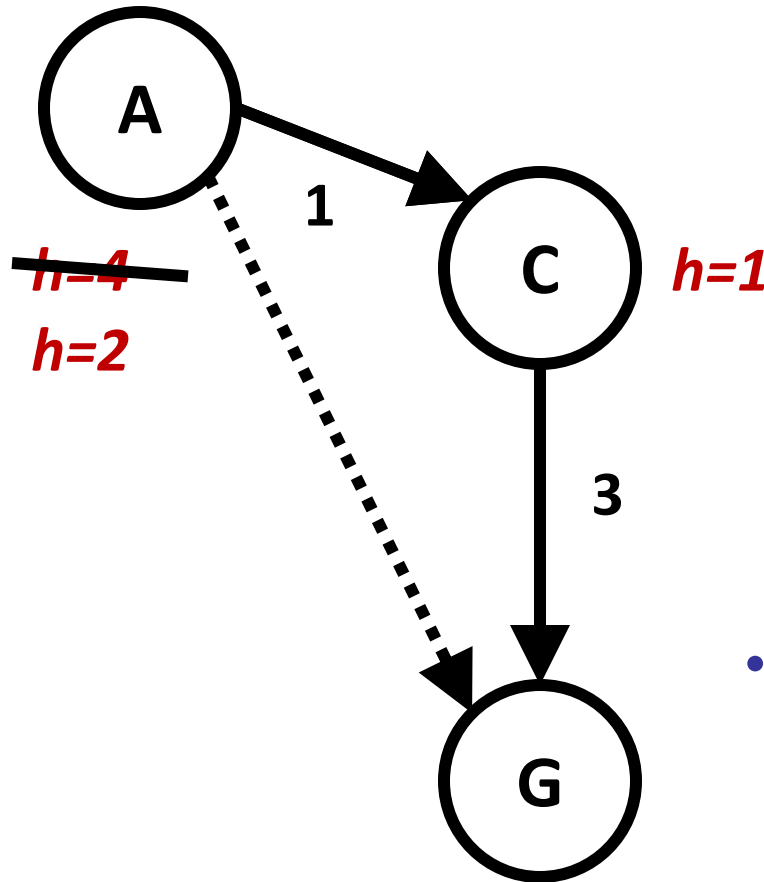
Informed Search



- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- Demo
- Creating Heuristics
- Tree Search
- **Graph Search**

Consistency of Heuristics



- Main idea: estimated heuristic costs $\leq$ actual costs
 - Admissibility: heuristic cost $\leq$ actual cost to goal
 $h(A) \leq \text{actual cost from A to G}$
 - Consistency: heuristic “arc” cost $\leq$ actual cost for each arc (stronger condition)
 $h(A) - h(C) \leq \text{cost}(A \text{ to } C)$
- Consequences of consistency:
 - The f value along a path never decreases
 $h(A) \leq \text{cost}(A \text{ to } C) + h(C)$
 - A* graph search is optimal



Informed Search

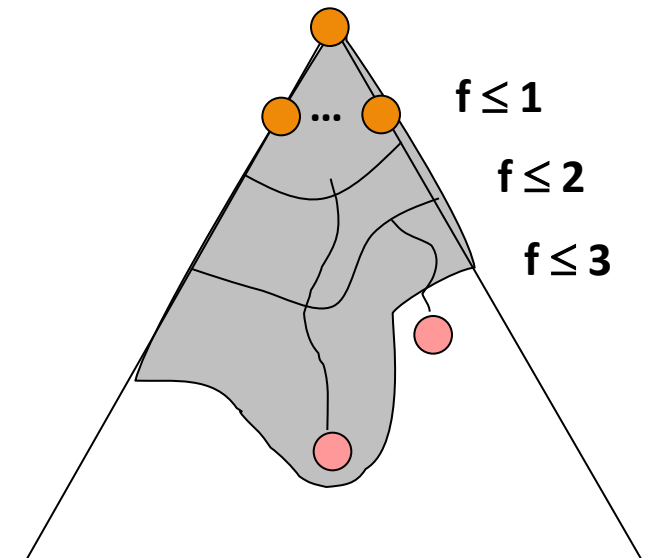


- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- Demo
- Creating Heuristics
- Tree Search
- **Graph Search**

Optimality of A* Graph Search

- Sketch: consider what A* does with a consistent heuristic:
 - Fact 1: In tree search, A* expands nodes in increasing total f value (f-contours)
 - Fact 2: For every state s, nodes that reach s optimally are expanded before nodes that reach s sub-optimally
 - Result: A* graph search is optimal



Informed Search



- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- Demo
- Creating Heuristics
- Tree Search
- Graph Search
- **Optimality**

Optimality

- Tree search:
 - A* is optimal if heuristic is admissible
 - UCS is a special case ($h = 0$)
- Graph search:
 - A* optimal if heuristic is consistent
 - UCS optimal ($h = 0$ is consistent)
- Consistency implies admissibility
- In general, most natural admissible heuristics tend to be consistent, especially if from relaxed problems



Informed Search

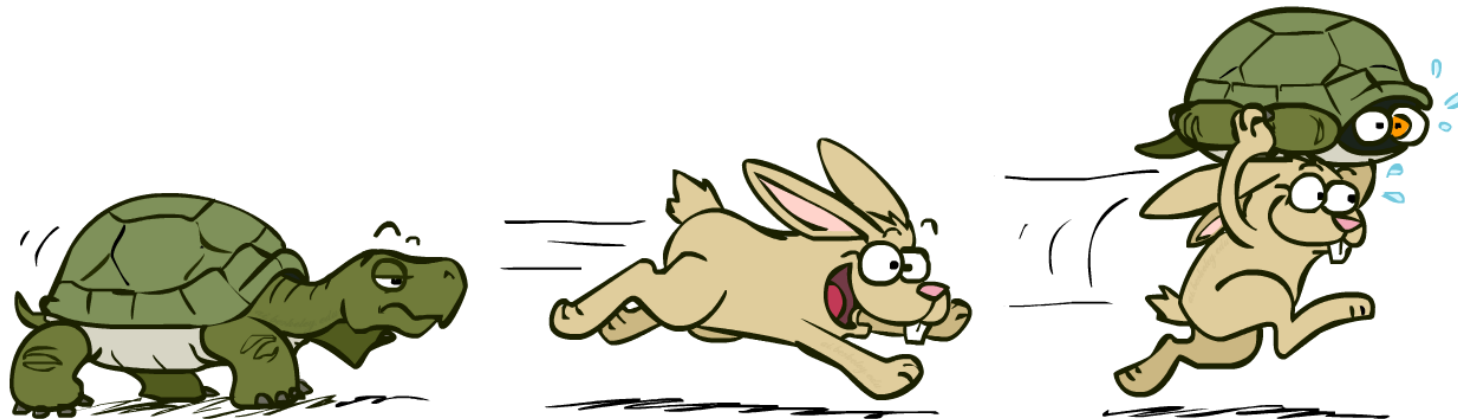


- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- Demo
- Creating Heuristics
- Tree Search
- Graph Search
- Optimality
- **A* Summary**

A* Summary

- A* uses both backward costs and (estimates of) forward costs
- A* is optimal with admissible / consistent heuristics
- Heuristic design is key: often use relaxed problems



Informed Search



- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- Demo
- Creating Heuristics
- Tree Search
- Graph Search
- Optimality
- A* Summary
- **Tree Search Pseudo Code**

Tree Search Pseudo Code

```
function TREE-SEARCH(problem, fringe) return a solution, or failure
  fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return node
    for child-node in EXPAND(STATE[node], problem) do
      fringe ← INSERT(child-node, fringe)
    end
  end
end
```



Informed Search



- **Informed Search**

- A* Search
- Admissible Heuristics
- Optimality of A* Tree Search
- Properties of A* Search
- UCS vs A* Search
- Demo
- Creating Heuristics
- Tree Search
- Graph Search
- Optimality
- A* Summary
- Tree Search Pseudo Code
- **Graph Search Pseudo Code**

Graph Search Pseudo Code

```
function GRAPH-SEARCH(problem, fringe) return a solution, or failure
    closed ← an empty set
    fringe ← INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
    loop do
        if fringe is empty then return failure
        node ← REMOVE-FRONT(fringe)
        if GOAL-TEST(problem, STATE[node]) then return node
        if STATE[node] is not in closed then
            add STATE[node] to closed
            for child-node in EXPAND(STATE[node], problem) do
                fringe ← INSERT(child-node, fringe)
            end
        end
    end
end
```

